JAVASCRIPT • MODULE 03

Arrays & Objects

Master JavaScript's Core Data Structures

■ DATA STRUCTURES

■ LEARNING OUTCOME

By the end of this module you will work fluently with arrays and objects using modern ES6+ syntax, use all array methods, apply destructuring, use Map and Set, and understand shallow vs deep copy.

■ Skills You Will Gain

- Create, access, and mutate arrays and objects
- Use all array methods: map, filter, reduce, find, sort
- Apply object destructuring, shorthand, spread, computed keys
- Use Map and Set for advanced data management
- Understand shallow vs deep copy
- Chain array methods confidently

■ 80% of real JavaScript programming is manipulating arrays and objects. Master these and you can solve almost any data problem -- in interviews and on the job.

SECTION 1

Arrays -- Full Reference

```
const nums = [1,2,3,4,5]; // ACCESS nums[0] // 1 first nums.at(-1) // 5 last (modern) nums.at(-2)
// 4 second to last // MUTATING methods (modify original) arr.push(6,7) // add to end arr.pop() //
remove from end arr.unshift(0) // add to start arr.shift() // remove from start arr.splice(1,2) //
remove 2 at index 1 arr.reverse() // reverse in place arr.sort((a,b)=>a-b)// numeric sort ascending
arr.sort((a,b)=>b-a)// numeric sort descending arr.fill(0,2,4) // fill indexes 2-3 with 0 //
NON-MUTATING methods (return new value) arr.slice(1,4) // [2,3,4] indexes 1-3 arr.concat([6,7]) //
new merged array arr.join(' - ') // '1 - 2 - 3' arr.indexOf(3) // 2 (index) or -1 arr.includes(3)
// true/false arr.find(n=>n>3) // 4 -- first match arr.findIndex(n=>n>3) // 3 -- index of first
arr.some(n=>n>4) // true -- at least one arr.every(n=>n>0) // true -- all match
[[1,2],[3,4]].flat() // [1,2,3,4] [1,[2,[3]]].flat(Infinity) // deeply flatten // CREATE
Array.from({length:5},(_,i)=>i+1) // [1,2,3,4,5] Array(5).fill(0) // [0,0,0,0,0]
Array.from('hello') // ['h','e','l','l','o'] [...new Set([1,2,2,3])] // [1,2,3] remove duplicates
```

SECTION 2

Objects -- Full Reference

```
const user = { name:'Alice', age:28, address:{ city:'Mumbai' } }; // ACCESS user.name // dot
notation user['email'] // bracket (dynamic keys) const k='age'; user[k] // 28 dynamic access //
ES6+ SHORTHAND const name='Bob'; const age=30; const short = { name, age }; // { name:'Bob', age:30
} // COMPUTED PROPERTY NAMES const prefix='user'; const obj = { [prefix+'Name']:'Carol',
[`${prefix}Id`]:42 }; // SPREAD const clone = { ...user }; // shallow copy const extended = {
...user, email:'x@y' }; // add property const updated = { ...user, age:29 }; // override property
// OBJECT METHODS Object.keys(user) // ['name','age','address'] Object.values(user) //
['Alice',28,{city:'Mumbai'}] Object.entries(user) // [['name','Alice'],['age',28],...]
Object.freeze(user) // prevent all modifications Object.assign({},a,b) // merge into {} (use spread
instead) // ITERATE entries for (const [key,val] of Object.entries(user)) { console.log(`${key}:
${val}`); } // DESTRUCTURING const { name:n, age:yr, address:{city} } = user; const { phone='N/A',
...rest } = user; // default + rest
```

SECTION 3

Map, Set, and Shallow vs Deep Copy

```
// MAP -- any type as key, preserves insertion order const map = new Map();
map.set('name','Alice'); map.set(42,'answer'); map.get('name') // 'Alice' map.has(42) // true
map.size // 2 map.delete('name') // true for (const [k,v] of map) console.log(k,'->',v); //
Frequency counter with Map function countWords(text) { const counts = new Map(); for (const word of
text.toLowerCase().split(' ')) { counts.set(word, (counts.get(word) || 0) + 1); } return counts; }
// SET -- unique values const set = new Set([1,2,3,2,1]); // {1,2,3} set.add(4); set.has(2);
set.delete(1); const arr = [...set]; // Set to array // Remove duplicates from array const unique =
[...new Set([1,2,2,3,3,4])]; // SHALLOW vs DEEP COPY const a = [1,2,3]; const b = a; // REFERENCE
-- same array! b.push(4); console.log(a); // [1,2,3,4] a was mutated! // Shallow copy (OK for flat
data) const c = [...a]; // spread const d = a.slice(); // Shallow PROBLEM with nested objects const
orig = { name:'Alice', scores:[90,85] }; const clone2 = { ...orig }; clone2.name = 'Bob'; // OK --
primitive, independent clone2.scores.push(95); // MUTATES orig.scores too! // DEEP COPY -- fully
independent const deep = structuredClone(orig); // modern, handles Date/Map/Set const deep2 =
JSON.parse(JSON.stringify(orig)); // simpler, no Date/Function
```

■ PRACTICE Q & A

Q1. What is the difference between arr.splice() and arr.slice()?

> ***A:*** *splice(): MUTATES original -- removes/inserts elements. slice(): NON-MUTATING -- returns new array with portion of original. splice(1,2) removes 2 at index 1. slice(1,3) returns indexes 1 and 2.*

Q2. When would you use Map instead of a plain object?

A: *Map when: keys are not strings, you need insertion-order iteration, you frequently add/remove keys, you need the .size property, or you use non-string keys.*

Q3. What is the difference between shallow copy and deep copy?

A: *Shallow: copies top level, nested objects still share references -- modifying nested properties affects both. Deep: recursively duplicates everything -- fully independent. Use structuredClone() for deep copy.*

Q4. How do you remove duplicates from an array?

A: *[...new Set(array)] -- convert to Set (removes duplicates) then spread back to array.*

Q5. What does arr.at(-1) do?

A: *Returns the last element of the array. Negative indices count from the end: at(-1)=last, at(-2)=second to last. More readable than arr[arr.length-1].*

■ Arrays & Objects Cheat Sheet	
`arr.push/pop`	Add/remove from end
`arr.shift/unshift`	Add/remove from start
`arr.slice(s,e)`	Copy portion -- non-mutating
`arr.splice(i,n)`	Remove n at index i -- mutates
`arr.sort((a,b)=>a-b)`	Numeric sort ascending
`arr.find(fn)`	First match or undefined
`arr.flat(Infinity)`	Deeply flatten
`[...new Set(arr)]`	Remove duplicates
`structuredClone(obj)`	Deep copy
`Object.entries(obj)`	Array of [key,val] pairs
`new Map()`	Key-value -- any key type
`new Set()`	Unique values